



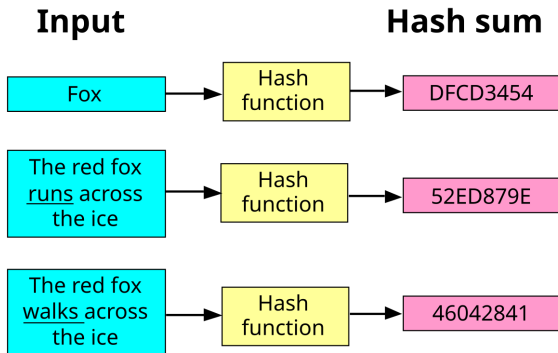
# Hashing & Commits

## **Corso di Laboratorio di Gestione progetto e organizzazione di impresa**

Prof. Dei Rossi, Leonardo Essam

## Che cos'è un hash? (1)

Un hash è il risultato di una funzione matematica (detta *funzione di hash*) che trasforma un insieme di dati di qualsiasi dimensione in una stringa di lunghezza fissa.



## Che cos'è un hash? (2)

Alcune caratteristiche fondamentali:

- **Lunghezza fissa:** anche se l'input è molto grande, l'hash ha sempre la stessa lunghezza (es. SHA-256 restituisce sempre 256 bit);
- **Deterministico:** lo stesso input produce sempre lo stesso hash;
- **Sensibilità agli input (avalanche effect):** basta cambiare un singolo bit dell'input e l'hash cambia completamente;
- **Difficile da invertire:** non puoi risalire facilmente all'input originale partendo dall'hash (funzione “one-way”);
- **Utile per verifiche:** confrontando hash di file o messaggi puoi controllare che i dati non siano stati modificati;

## Commit su Git (1)

Dalle lezioni precedenti, ricordiamo che un commit su Git contiene:

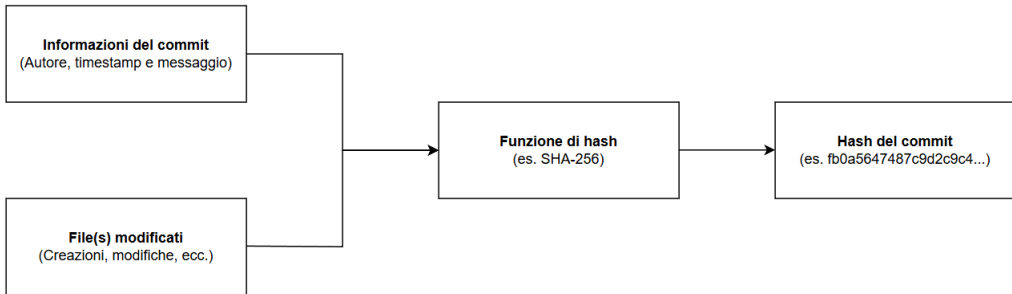
1. Autore (nome, cognome ed e-mail);
2. Timestamp (data e ora) del commit;
3. Messaggio personalizzato.

L'insieme di queste informazioni, più il contenuto dei file effettivamente modificati, genera il *payload* per la funzione di hash<sup>1</sup>.

---

<sup>1</sup><https://stackoverflow.com/questions/35430584/how-is-the-git-hash-calculated>

## Commit su Git (2)



## Padri e figlie (1)

Per costruire lo storico dei commit vi è la necessità di includere il commit "genitore" (o in caso del primo commit, l'hash della radice dell'albero).

Di seguito lo schema dell'intestazione di un commit:

```
1 tree <hash-tree>
2 parent <hash-commit-genitore>
3 author Amy <amy@example.com> 1695299977 +0200
4 committer Chris <chris@example.com> 1695299977 +0200
5
6 Messaggio del commit
```

## Padri e figlie (2)

Osserviamo però che ci sono due campi che sembrano uguali:

```
1 [...]
2 author Amy <amy@example.com> 1695299977 +0200
3 committer Chris <chris@example.com> 1695299977 +0200
4 [...]
```

Scopriremo più avanti nel corso, quando parleremo di GitHub, che non sempre l'autore della modifica in locale è lo stesso autore della modifica al progetto condiviso con tutti.

**Per ora fidatevi del prof. e tenete a mente solo il *committer* :)**

## La macchina del tempo (1)

Come detto nella prima lezione, Git ha una "macchina del tempo" integrata.

A parte gli scherzi, una delle funzionalità principali di Git è quella di poter muoversi nello storico dei commit del progetto per poter ripristinare delle modifiche di cui magari si è fatto il commit per errore.



## La macchina del tempo (2)

Apreno Git Bash, digitare i seguenti comandi:

```
git clone https://github.com/leonardodeirossi/prova-5inb
cd prova-5inb
git log
```

Osservare l'output del terminale (il primo commit):

```
leode@ThinkBook-14:~/prova-5inb$ git log
commit 8ab0d672883a239c2c187d18c466bdaddf8d0cc0 (HEAD -> main, origin/main, origin/HEAD)
Author: Leonardo Essam <leonardoessam.deirossi@gmail.com>
Date:   Sun Sep 21 14:16:55 2025 +0200

    Ho ri-aggiunto la riga eliminata e una nuova riga
```

## La macchina del tempo (3)

Osserviamo ora con attenzione la parola HEAD (evidenziata):

```
leode@ThinkBook-14:~/prova-5inb$ git log
commit 8ab0d672883a239c2c187d18c466bdaddf8d0cc0 (HEAD -> main, origin/main, origin/HEAD)
Author: Leonardo Essam <leonardoessam.deirossi@gmail.com>
Date:   Sun Sep 21 14:16:55 2025 +0200

    Ho ri-aggiunto la riga eliminata e una nuova riga
```

Quella *keyword* (parola chiave) indica la posizione in cui ci troviamo all'interno dell'albero<sup>2</sup>.

---

<sup>2</sup>O, volendo, all'interno del *continuum spazio temporale*.

## La macchina del tempo (4)

Supponiamo ora di voler vedere com'era fatto il file README.md prima che venisse ri-aggiunta la riga eliminata e aggiunta una riga nuova (vedasi messaggio dell'ultimo commit).

Dal terminale osserviamo il commit subito seguente (quindi subito precedente nel tempo):

```
commit 03614e6ed32d18e778e961ad3225bd25c49f31ac
Author: Leonardo Essam <leonardoessam.deirossi@gmail.com>
Date:   Sun Sep 21 14:16:07 2025 +0200

    Ho eliminato una riga
```

## La macchina del tempo (5)

Come detto prima, l'hash del commit è una stringa alfanumerica che identifica univocamente il commit all'interno del progetto.

Per poter tornare indietro, e di conseguenza spostare l'HEAD, vale il comando:

```
git checkout <hash del commit>
```

Ovvero, in questo caso:

```
git checkout 03614e6ed32d18e778e961ad3225bd25c49f31ac
```

## La macchina del tempo (6)

Osserviamo che il terminale restituisce una serie di lamentele, ma per ora possiamo ignorarle.

Ora, aprendo il file README.md (da terminale con `cat README.md` o con VS Code), osserviamo che il contenuto è cambiato rispetto a prima.

Provando ora ad eseguire il comando `git log`, notiamo che il commit dal quale siamo partiti non c'è più. Ma perché?

## La macchina del tempo (7)

Visto che abbiamo fatto un "viaggio nel tempo", Git si è "dimenticato" dei commit successivi a quello in cui ci siamo spostati.

Ovviamente, in realtà non sono andati persi, si può sempre tornare al punto più recente con:

```
git checkout main
```

**Piccola osservazione:** nelle versioni più recenti di Git/GitHub il comando funziona con la parola `main`, mentre nelle versioni meno recenti è necessaria la vecchia parola `master`.

